Compte rendu de TP : Développement d'applications Android avec IHM en XML

Filière : ILCS – S3 | Module : Développement Mobile | TP : Partie 3

Réalisé par : Hanane El Aasraoui | Date : 14/10/2026

1. **Introduction**

   Android sépare la description de l'interface (fichiers XML de res/layout) de la logique de l'application (code Java). Ce TP applique cette approche à trois exercices : périmètre d'un cercle, convertisseur de devises (MAD, USD, EUR) et formulaire Étudiant (classes Personne et Etudiant).

2. **Objectifs du TP**

   • Décrire une interface utilisateur dans un fichier XML (TextView, EditText, Button, RadioGroup, Spinner, ImageView).

• Récupérer les composants dans le code Java avec findViewById et réagir aux événements avec setOnClickListener.
• Mettre en forme un texte (police, taille, couleur) et afficher un résultat avec un Toast.
• Utiliser les ressources du projet : res/drawable (images), res/values (tableaux de chaînes).
• Appliquer la programmation orientée objet (classes, constructeurs, accesseurs, héritage) dans une application Android.

3. Rôle des dossiers et fichiers du projet

Un composant XML reçoit un identifiant avec android:id="@+id/nom". Dans le code Java, il est référencé par R.id.nom. De même, setContentView(R.layout.activity_main) affiche le fichier activity_main.xml.

res/layout/*.xml

Décrit l'interface de l'activité (TextView, EditText, Button, ...)

res/values/

Contient les textes, couleurs, dimensions et tableaux de chaînes (string-array du Spinner)

res/drawable/

Contient les images utilisées par l'interface (logos des devises)

MainActivity.java

Logique de l'application : récupère les composants, réagit aux clics, calcule

AndroidManifest.xml

Déclare l'application et son activité principale

build.gradle.kts (:app)

Configuration Gradle et dépendances (androidx.appcompat)

4.Exercice 1 : Périmètre d'un cercle

4.1 Énoncé Calculer le périmètre d'un cercle ($2 \times \pi \times R$, $\pi \approx 3,14$) avec une interface XML : titre stylé, champ de saisie du rayon R et résultat affiché dans un Toast.

res/layout/activity_main.xml

Décrit l'interface : titre stylé, champ de saisie R, bouton Calculer

MainActivity.java

Lit R, calcule $2 \times \pi \times R$ et affiche le résultat dans un Toast

AndroidManifest.xml

Déclare MainActivity comme activité de démarrage (généré automatiquement)

4.2 Interface : activity_main.xml

COUDE SOURCE:

<TextView android:text="Périmètre de cercle:"
android:textSize="24sp"

android:textStyle="bold" android:textColor="#555555"
android:fontFamily="serif" />

<EditText android:id="@+id/editR"
android:inputType="numberDecimal" />

<Button android:id="@+id/btnCalc"
android:text="Calculer" />

Chaque attribut du TextView règle un aspect du style : taille, gras, couleur et police. L'attribut android:id permet de retrouver le champ et le bouton dans le code Java.

4.3 Code Java : MainActivity.java

btnCalc.setOnClickListener(v -> {

double r = Double.parseDouble(editR.getText().toString());
double p = 2 * 3.14 * r;

Toast.makeText(this, "Périmètre " + p, Toast.LENGTH_LONG).show(); });

Au clic, le programme lit R, calcule 2 × π × R et affiche le résultat dans un Toast

5.Exercice 2 : Convertisseur de devises

5.1 Énoncé Convertir simultanément MAD, USD et EUR : l'utilisateur saisit un montant dans un champ, le bouton Convertir remplit les deux autres et un second bouton remet les champs à 0. Les logos sont placés dans res/drawable

res/layout/activity_main.xml

Trois lignes (logo + champ de saisie) et deux boutons **res/drawable/ic_mad, ic_usd, ic_eu**r

Logos des trois devises

MainActivity.java

Mémorise le champ actif, convertit les montants, remet les champs à 0

5.2 Code Java : MainActivity.java

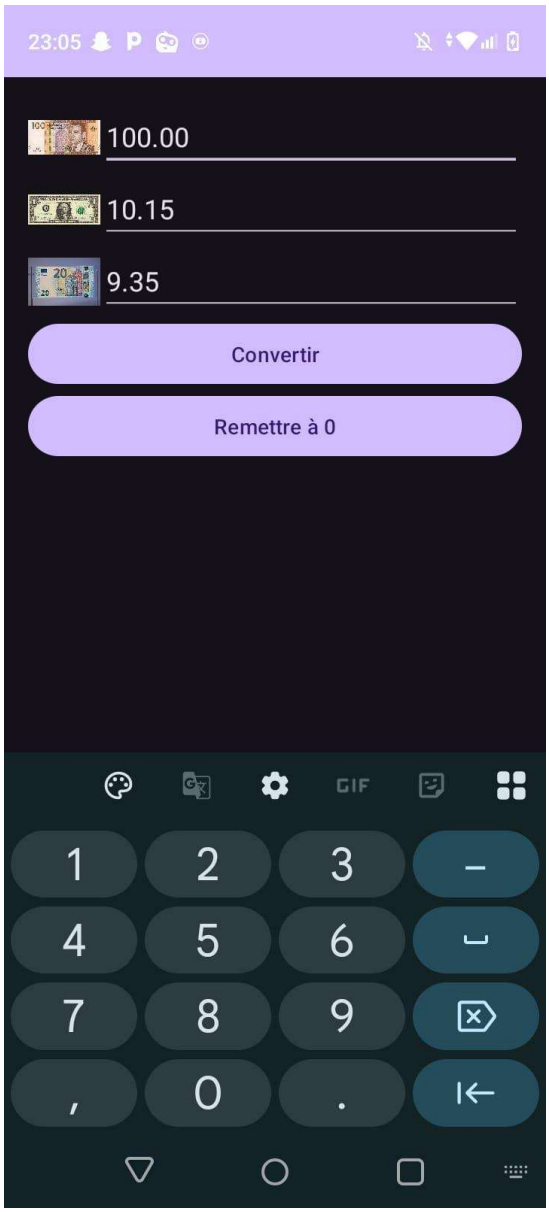
static final double USD_MAD = 9.85; static final double EUR_MAD = 10.69;

void convertir() { double mad; if (source == 1) mad = lire(edtUsd) * USD_MAD; else if (source == 2) mad = lire(edtEur) * EUR_MAD; else mad = lire(edtMad);

```
edtMad.setText(String.format(Locale.US,
"%.2f", mad));
edtUsd.setText(String.format(Locale.US,
"%.2f", mad / USD_MAD));
edtEur.setText(String.format(Locale.US,
"%.2f", mad / EUR_MAD));
```

} Le champ actif (source) est mémorisé, le montant est converti en MAD puis dans les deux autres devises. Locale.US impose le point décimal, sinon un téléphone en français écrit une virgule et la conversion suivante échoue. Pour mettre à jour les taux, il suffit de modifier les deux constantes.

5.3 TEST



6. Exercice 3 : Classes Personne et Etudiant et formulaire

6.1 Énoncé

Écrire les classes Personne (prenom, nom, eMail) et Etudiant (annee, matiere, moyenne) qui hérite de

Personne, un formulaire XML avec bouton Enregistrer, puis afficher les données de l'étudiant en HTML dans un

Personne.java

Classe de base : prénom, nom, eMail, constructeurs, accesseurs, toString()

Etudiant.java

Hérite de Personne et ajoute année, matière, moyenne

res/values/string-array.xml

Liste des matières affichée dans le Spinner

res/layout/content_etudiant_main.xml

Formulaire : EditText, RadioGroup (1A / 2A), Spinner, bouton Enregistrer

MainActivity.java

Crée l'objet Etudiant et affiche le HTML dans un Toast

6.2 Classe Personne public class Personne { private String prenom, nom, eMail;

```
public Personne() { prenom = ""; nom = "";
eMail = ""; }
public Personne(String prenom, String nom,
String eMail) {
```

```
    this.prenom = prenom; this.nom = nom;
this.eMail = eMail;
}
public String toString() {
    return "Prenom: " + prenom + " Nom: " +
nom + " eMail: " + eMail;
}
// getNom(), setNom(), getPrenom(),
setPrenom(), getMail(), setMail()


}
```

Les attributs sont privés (encapsulation) et accessibles uniquement par les accesseurs et modifieurs. Le second constructeur est une surcharge du premier.

6.3 Classe Etudiant (héritage)

public class Etudiant extends Personne { private int annee; private String matiere; private double moyenne;

```
public Etudiant() { super(); }
public Etudiant(String prenom, String nom,
String eMail,
                int annee, String matiere,
double moyenne) {
    super(prenom, nom, eMail);
    this.annee = annee; this.matiere =
matiere; this.moyenne = moyenne;
}
```

```
// getAnnee(), setAnnee(), getMatiere(),
setMatiere(), getMoyenne(), setMoyenne()


}
```

extends Personne indique que Etudiant hérite des attributs et des méthodes de Personne. super(prenom, nom, eMail) appelle le constructeur de la classe mère.

6.4 Traitement dans MainActivity.java

// Clic sur Enregistrer : création de l'objet puis affichage du HTML

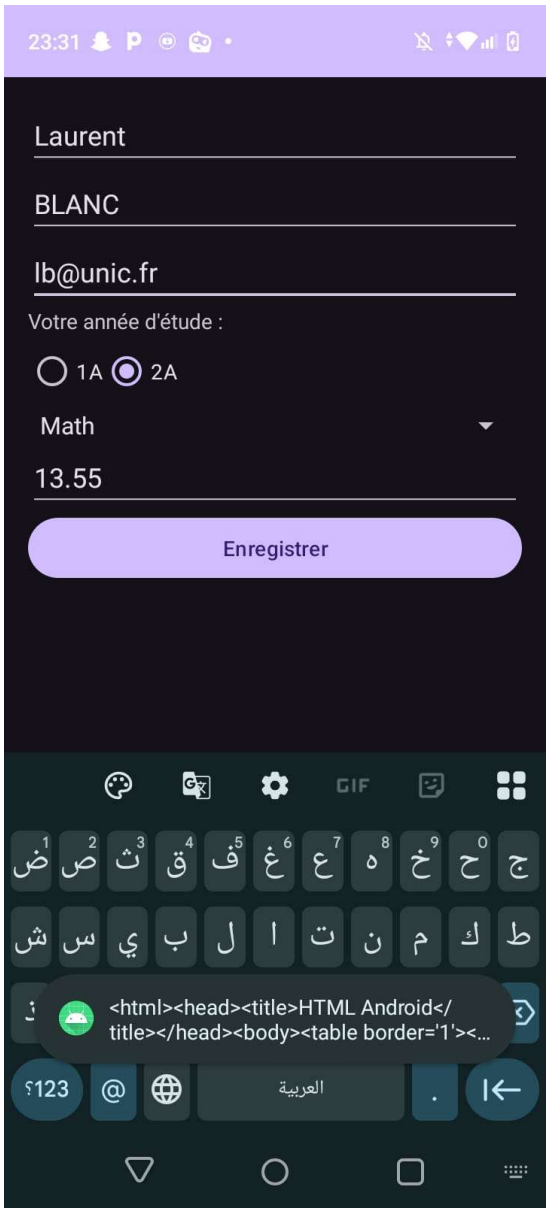
etudiant = new Etudiant(prenom, nom, mail, annee, matiere, moyenne);

Toast.makeText(this, genererHtml(), Toast.LENGTH_LONG).show();

// genererHtml() construit la chaîne : <html>...<table border='1'>...</table></html>

Le Spinner est rempli depuis le string-array avec un ArrayAdapter, l'année vient du RadioGroup (getCheckedRadioButtonId) et genererHtml() assemble un tableau HTML avec les six valeurs de l'étudiant.

6.5 Test



7. Conclusion

Ce TP a permis de mettre en pratique la conception d'interfaces Android décrites en XML et leur liaison avec le code Java. Le premier exercice a introduit les bases (TextView, EditText, Button, Toast). Le deuxième a ajouté la gestion des ressources drawable, des écouteurs d'événements et des calculs de conversion. Le troisième a

combiné programmation orientée objet (héritage, encapsulation) et composants de formulaire (RadioGroup, Spinner).